



Universidad Nacional de Colombia
Sede Medellín

Programación Orientada a Objetos

Yuliany Cifuentes Rondón
c.c 100295231

Docente: Walter Hugo Arboleda Mazo

Ejercicios Actividad 2
Repositorio GitHub:

<https://github.com/ycifuentesr-code/Trabajos-programaci-n-orientada-a-objetos/blob/main/README.md>

Ejercicios Actividad 2

Ejercicio 2.1.

Se requiere un programa que modele el concepto de una persona. Una persona posee nombre, apellido, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice los valores de sus respectivos atributos. La clase debe incluir los siguientes métodos:

- Definir un método que imprima en pantalla los valores de los atributos del objeto.
- En un método main se deben crear dos personas y mostrar los valores de sus atributos en pantalla.

Python

```
"""
Ejercicio 2.1 - Definición de clases
Clase Persona: modela una persona con nombre, apellidos,
número de documento de identidad y año de nacimiento.
"""

class Persona:
    """Define objetos de tipo Persona."""

    def __init__(self, nombre: str, apellidos: str,
                  numero_documento_identidad: str,
                  anio_nacimiento: int):
        """Constructor: inicializa los atributos de la
        persona."""
        self.nombre = nombre
        self.apellidos = apellidos
        self.numero_documento_identidad =
numero_documento_identidad
        self.anio_nacimiento = anio_nacimiento

    def imprimir(self) -> None:
        """Imprime en pantalla los valores de los atributos."""
        print(f"Nombre = {self.nombre}")
        print(f"Apellidos = {self.apellidos}")
```

```

        print(f"Número de documento de identidad =
{self.numero_documento_identidad}")
        print(f"Año de nacimiento = {self.anio_nacimiento}")
        print()

def main():
    """Crea dos personas y muestra sus datos."""
    p1 = Persona("Pedro", "Pérez", "1053121010", 1998)
    p2 = Persona("Luis", "León", "1053223344", 2001)
    p1.imprimir()
    p2.imprimir()

if __name__ == "__main__":
    main()

```

Resultado del código ejecutado

```

... Nombre = Pedro
    Apellidos = Pérez
    Número de documento de identidad = 1053121010
    Año de nacimiento = 1998

    Nombre = Luis
    Apellidos = León
    Número de documento de identidad = 1053223344
    Año de nacimiento = 2001

```

Diagrama de clases

Persona
+ nombre: str + apellidos: str + numero_documento_identidad: str + anio_nacimiento: int
«constructor» __init__(nombre: str, apellidos: str, numero_documento_identidad: str, anio_nacimiento: int) + imprimir(): None

URL: <https://colab.research.google.com/drive/1Y9c6OLCPHW8wEkTgN7E8zKusKHgloXFq>

Ejercicio 2.2. Definición de atributos de una clase con tipos primitivos de datos

Las clases contienen una colección de atributos, que representan propiedades de los objetos. Dichos atributos tienen un tipo, el cual puede ser un tipo primitivo de dato de Java u objeto. En la tabla 2.2 se presentan los tipos primitivos de datos en Java (Arroyo-Díaz, 2019a).

Python

"""

Ejercicio 2.2 - Definición de atributos de una clase con tipos primitivos de datos

Clase Planeta: modela un planeta del sistema solar.

"""

from enum import Enum

class TipoPlaneta(Enum):

"""Tipo de planeta según su tamaño (equivale al enum de Java)."""

GASEOSO = "GASEOSO"

TERRESTRE = "TERRESTRE"

ENANO = "ENANO"

class Planeta:

"""Define objetos de tipo Planeta."""

Unidad astronómica: distancia Tierra-Sol en km

UA = 149_597_870

def __init__(self, nombre: str = None, cantidad_satelites: int = 0, masa: float = 0.0, volumen: float = 0.0, diametro: int = 0, distancia_sol: int = 0, tipo: TipoPlaneta = None, es_observable: bool = False):

"""Constructor. Los valores por defecto son los valores iniciales que pide el enunciado (null -> None, 0, 0.0 y False).

:param masa: masa en kilogramos

:param volumen: volumen en kilómetros cúbicos

:param diametro: diámetro en kilómetros

:param distancia_sol: distancia media al Sol en millones de km

"""

self.nombre = nombre

self.cantidad_satelites = cantidad_satelites

self.masa = masa

```

        self.volumen = volumen
        self.diametro = diametro
        self.distancia_sol = distancia_sol
        self.tipo = tipo
        self.es_observable = es_observable

def imprimir(self) -> None:
    """Imprime los valores de los atributos del planeta."""
    print(f"Nombre del planeta = {self.nombre}")
    print(f"Cantidad de satélites = {self.cantidad_satelites}")
    print(f"Masa del planeta = {self.masa} kg")
    print(f"Volumen del planeta = {self.volumen} km³")
    print(f"Diámetro del planeta = {self.diametro} km")
    print(f"Distancia al Sol = {self.distancia_sol} millones de km")
    print(f"Tipo de planeta = {self.tipo.value if self.tipo else None}")
    print(f"Es observable = {self.es_observable}")

def calcular_densidad(self) -> float:
    """Devuelve la densidad (masa / volumen) en kg/km³."""
    if self.volumen == 0:
        return 0.0
    return self.masa / self.volumen

def es_planeta_exterior(self) -> bool:
    """Un planeta es exterior si está más allá del cinturón de
    asteroides (que termina en 3.4 UA)."""
    limite_km = Planeta.UA * 3.4
    distancia_km = self.distancia_sol * 1_000_000 # millones de km -> km
    return distancia_km > limite_km

def main():
    """Crea dos planetas, imprime sus datos, su densidad y si son
    exteriores."""
    p1 = Planeta("Tierra", 1, 5.9736e24, 1.08321e12, 12742, 150,
                 TipoPlaneta.TERRESTRE, True)
    p1.imprimir()
    print(f"Densidad del planeta = {p1.calcular_densidad():.4e} kg/km³")
    print(f"Es planeta exterior = {p1.es_planeta_exterior()}")
    print()

    p2 = Planeta("Júpiter", 79, 1.899e27, 1.4313e15, 139820, 750,
                 TipoPlaneta.GASEOSO, True)
    p2.imprimir()
    print(f"Densidad del planeta = {p2.calcular_densidad():.4e} kg/km³")
    print(f"Es planeta exterior = {p2.es_planeta_exterior()}")

```

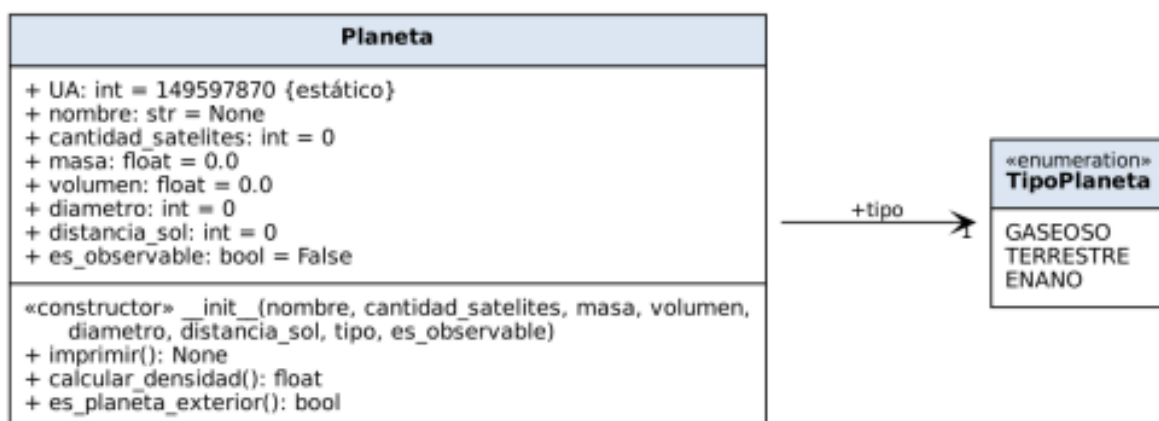
```
if __name__ == "__main__":
    main()
```

Resultado del código ejecutado

```
... Nombre del planeta = Tierra
Cantidad de satélites = 1
Masa del planeta = 5.9736e+24 kg
Volumen del planeta = 10832100000000.0 km³
Diámetro del planeta = 12742 km
Distancia al Sol = 150 millones de km
Tipo de planeta = TERRESTRE
Es observable = True
Densidad del planeta = 5.5147e+12 kg/km³
Es planeta exterior = False

Nombre del planeta = Júpiter
Cantidad de satélites = 79
Masa del planeta = 1.899e+27 kg
Volumen del planeta = 1431300000000000.0 km³
Diámetro del planeta = 139820 km
Distancia al Sol = 750 millones de km
Tipo de planeta = GASEOSO
Es observable = True
Densidad del planeta = 1.3268e+12 kg/km³
Es planeta exterior = True
```

Diagrama de clase



URL:

<https://colab.research.google.com/drive/1a3nMXtITNAeWFeDy4pmie-1SY4kpTaWg#scrollTo=k4JedmdxcLU9>

Ejercicio 2.3. Estado de un objeto

Se denomina estado de un objeto al conjunto de pares {atributo = valor de un objeto}. El estado de un objeto puede cambiar a lo largo de su vida a medida que se vayan ejecutando sus métodos (Arroyo-Díaz, 2019a). Las clases tienen dos tipos de métodos: get y set.

- Los métodos get permiten obtener el valor de un atributo de un objeto. Los métodos get se definen con el siguiente formato:

getNombreAtributo

- Los métodos set permiten asignar o cambiar el valor de un atributo de un objeto y, por lo tanto, cambian su estado. Los métodos set se definen con el siguiente formato:

setNombreAtributo(tipo parámetro)

Los métodos get y set tienen una estructura que se repite para cada uno de los atributos. Por ello, los entornos de desarrollo actuales permiten generar automáticamente dichos métodos.

Python

```
"""
```

Ejercicio 2.3 - Estado de un objeto

Clase Automovil con métodos get y set, acelerar, desacelerar, frenar y calcular tiempo de llegada.

```
"""
```

```
from enum import Enum
```

```
class TipoCombustible(Enum):
```

```
    GASOLINA = "Gasolina"
```

```
    BIOETANOL = "Bioetanol"
```

```
    DIESEL = "Diésel"
```

```
    BIODIESEL = "Biodiésel"
```

```
    GAS_NATURAL = "Gas natural"
```

```
class TipoAutomovil(Enum):
```

```
    CIUDAD = "Carro de ciudad"
```

```
    SUBCOMPACTO = "Subcompacto"
```

```
    COMPACTO = "Compacto"
```

```
    FAMILIAR = "Familiar"
```

```
    EJECUTIVO = "Ejecutivo"
```

```
    SUV = "SUV"
```

```

class Color(Enum):
    BLANCO = "Blanco"
    NEGRO = "Negro"
    ROJO = "Rojo"
    NARANJA = "Naranja"
    AMARILLO = "Amarillo"
    VERDE = "Verde"
    AZUL = "Azul"
    VIOLETA = "Violeta"

class Automovil:
    """Define objetos de tipo Automovil."""

    def __init__(self, marca: str, modelo: int, motor: float,
                  tipo_combustible: TipoCombustible, tipo_automovil:
TipoAutomovil,
                  numero_puertas: int, cantidad_asientos: int,
                  velocidad_maxima: int, color: Color):
        self.marca = marca
        self.modelo = modelo                # año de fabricación
        self.motor = motor                  # cilindraje en litros
        self.tipo_combustible = tipo_combustible
        self.tipo_automovil = tipo_automovil
        self.numero_puertas = numero_puertas
        self.cantidad_asientos = cantidad_asientos
        self.velocidad_maxima = velocidad_maxima # km/h
        self.color = color
        self.velocidad_actual = 0           # inicia detenido

    # ----- Métodos get -----
    def get_marca(self) -> str:
        return self.marca

    def get_modelo(self) -> int:
        return self.modelo

    def get_motor(self) -> float:
        return self.motor

    def get_tipo_combustible(self) -> TipoCombustible:
        return self.tipo_combustible

    def get_tipo_automovil(self) -> TipoAutomovil:
        return self.tipo_automovil

```

```

def get_numero_puertas(self) -> int:
    return self.numero_puertas

def get_cantidad_asientos(self) -> int:
    return self.cantidad_asientos

def get_velocidad_maxima(self) -> int:
    return self.velocidad_maxima

def get_color(self) -> Color:
    return self.color

def get_velocidad_actual(self) -> int:
    return self.velocidad_actual

# ----- Métodos set -----
def set_marca(self, marca: str) -> None:
    self.marca = marca

def set_modelo(self, modelo: int) -> None:
    self.modelo = modelo

def set_motor(self, motor: float) -> None:
    self.motor = motor

def set_tipo_combustible(self, tipo_combustible: TipoCombustible) -> None:
    self.tipo_combustible = tipo_combustible

def set_tipo_automovil(self, tipo_automovil: TipoAutomovil) -> None:
    self.tipo_automovil = tipo_automovil

def set_numero_puertas(self, numero_puertas: int) -> None:
    self.numero_puertas = numero_puertas

def set_cantidad_asientos(self, cantidad_asientos: int) -> None:
    self.cantidad_asientos = cantidad_asientos

def set_velocidad_maxima(self, velocidad_maxima: int) -> None:
    self.velocidad_maxima = velocidad_maxima

def set_color(self, color: Color) -> None:
    self.color = color

def set_velocidad_actual(self, velocidad_actual: int) -> None:
    self.velocidad_actual = velocidad_actual

# ----- Métodos que cambian el estado -----
def acelerar(self, incremento_velocidad: int) -> None:

```

```

        """Aumenta la velocidad sin superar la velocidad máxima."""
        if self.velocidad_actual + incremento_velocidad <=
self.velocidad_maxima:
            self.velocidad_actual += incremento_velocidad
        else:
            print("No se puede incrementar a una velocidad superior "
                  "a la máxima del automóvil.")

def desacelerar(self, decremento_velocidad: int) -> None:
    """Disminuye la velocidad sin llegar a un valor negativo."""
    if self.velocidad_actual - decremento_velocidad >= 0:
        self.velocidad_actual -= decremento_velocidad
    else:
        print("No se puede decrementar a una velocidad negativa.")

def frenar(self) -> None:
    """Coloca la velocidad actual en cero."""
    self.velocidad_actual = 0

def calcular_tiempo_llegada(self, distancia: float) -> float:
    """Tiempo estimado (horas) = distancia (km) / velocidad actual
(km/h)."""
    if self.velocidad_actual == 0:
        print("El automóvil está detenido: no se puede estimar el tiempo.")
        return float("inf")
    return distancia / self.velocidad_actual

def imprimir(self) -> None:
    print(f"Marca = {self.marca}")
    print(f"Modelo = {self.modelo}")
    print(f"Motor = {self.motor} L")
    print(f"Tipo de combustible = {self.tipo_combustible.value}")
    print(f"Tipo de automóvil = {self.tipo_automovil.value}")
    print(f"Número de puertas = {self.numero_puertas}")
    print(f"Cantidad de asientos = {self.cantidad_asientos}")
    print(f"Velocidad máxima = {self.velocidad_maxima} km/h")
    print(f"Color = {self.color.value}")

def main():
    auto1 = Automovil("Ford", 2018, 3.0, TipoCombustible.DIESEL,
                      TipoAutomovil.EJECUTIVO, 5, 6, 250, Color.NEGRO)
    auto1.imprimir()

    auto1.set_velocidad_actual(100)
    print(f"Velocidad actual = {auto1.get_velocidad_actual()} km/h")
    print(f"Tiempo para recorrer 300 km = "
          f"{auto1.calcular_tiempo_llegada(300):.2f} h")

```

```

auto1.acelerar(20)
print(f"Velocidad actual = {auto1.get_velocidad_actual()} km/h")

auto1.desacelerar(50)
print(f"Velocidad actual = {auto1.get_velocidad_actual()} km/h")

auto1.frenar()
print(f"Velocidad actual = {auto1.get_velocidad_actual()} km/h")

auto1.desacelerar(20) # muestra el mensaje de error (velocidad negativa)

if __name__ == "__main__":
    main()

```

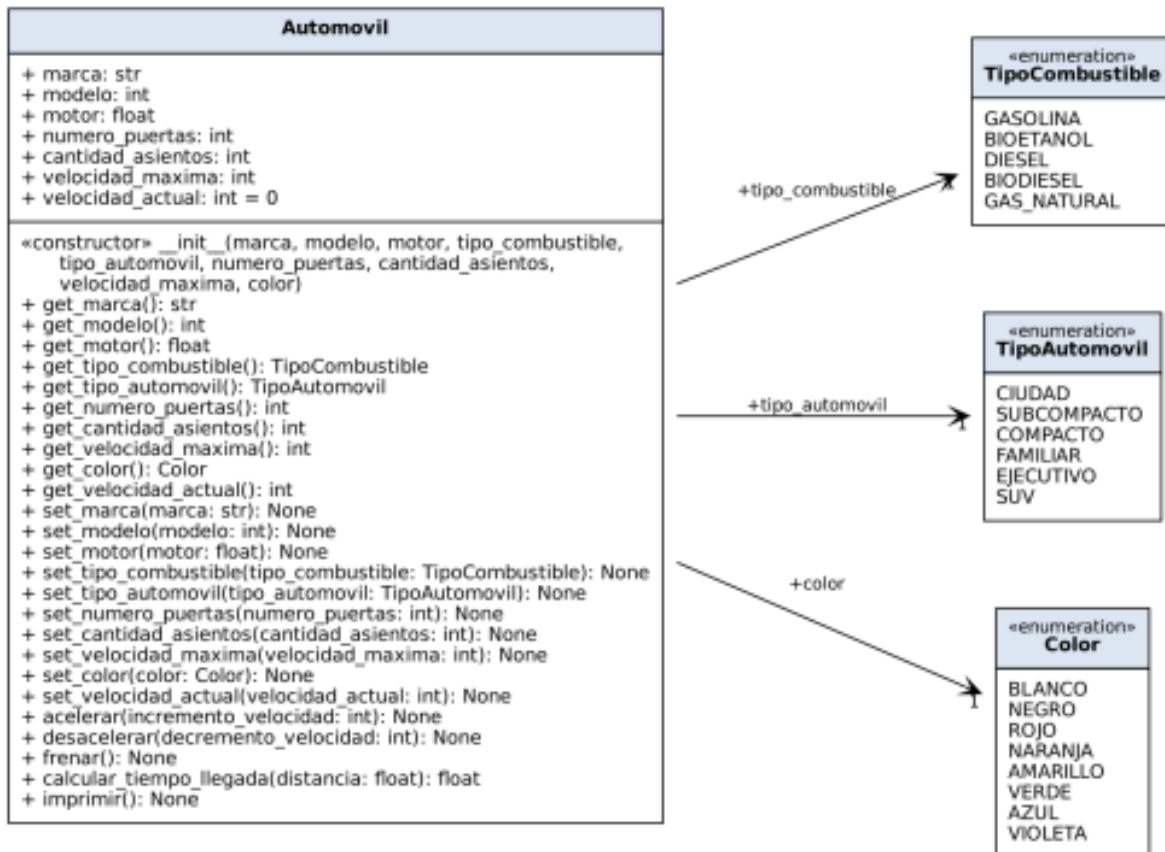
Resultado del código ejecutado

```

...  Marca = Ford
      Modelo = 2018
      Motor = 3.0 L
      Tipo de combustible = Diésel
      Tipo de automóvil = Ejecutivo
      Número de puertas = 5
      Cantidad de asientos = 6
      Velocidad máxima = 250 km/h
      Color = Negro
      Velocidad actual = 100 km/h
      Tiempo para recorrer 300 km = 3.00 h
      Velocidad actual = 120 km/h
      Velocidad actual = 70 km/h
      Velocidad actual = 0 km/h
      No se puede decrementar a una velocidad negativa.

```

Diagrama de clase



URL: https://colab.research.google.com/drive/1DRJQMoeUt0U9-0KfYJx83y5AkCdkN_I4

Ejercicio 2.4. Definición de métodos con y sin valores de retorno

Los métodos de una clase con y sin valores de retorno tienen el siguiente formato, el cual se denomina **signatura del método** (Deitel y Deitel, 2017):

tipoRetorno nombreMétodo(tipo parámetro1, tipo parámetro2, ...)

- **tipoRetorno:** representa el tipo de retorno del método, el cual puede ser un tipo de dato primitivo, un objeto, o si no devuelve nada debe colocarse la palabra reservada `void`.
- **nombreMétodo:** representa el nombre del método.
- **Parámetros:** es un listado de parámetros separados por comas, que se utilizan en el cuerpo del método para cumplir la finalidad del método. Para cada parámetro se debe colocar previamente su tipo de dato. Un método puede no tener parámetros.

Python

```
"""
```

Ejercicio 2.4 - Definición de métodos con y sin valores de retorno
Clases `Circulo`, `Rectangulo`, `Cuadrado` y `TrianguloRectangulo`,
y una clase de prueba `PruebaFiguras` con el método `main`.

```
"""
```

```
import math
```

```

class Circulo:
    def __init__(self, radio: float):
        self.radio = radio # en cm

    def calcular_area(self) -> float:
        return math.pi * self.radio ** 2

    def calcular_perimetro(self) -> float:
        return 2 * math.pi * self.radio

class Rectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base
        self.altura = altura

    def calcular_area(self) -> float:
        return self.base * self.altura

    def calcular_perimetro(self) -> float:
        return 2 * self.base + 2 * self.altura

class Cuadrado:
    def __init__(self, lado: float):
        self.lado = lado

    def calcular_area(self) -> float:
        return self.lado ** 2

    def calcular_perimetro(self) -> float:
        return 4 * self.lado

class TrianguloRectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base
        self.altura = altura

    def calcular_area(self) -> float:
        return self.base * self.altura / 2

    def calcular_hipotenusa(self) -> float:
        """Teorema de Pitágoras:  $h = \sqrt{base^2 + altura^2}$ ."""
        return math.sqrt(self.base ** 2 + self.altura ** 2)

```

```

def calcular_perimetro(self) -> float:
    # Un método puede reutilizar otro método de la misma clase
    return self.base + self.altura + self.calcular_hipotenusa()

def determinar_tipo_triangulo(self) -> None:
    """Método SIN valor de retorno: imprime el tipo de triángulo."""
    a, b, c = self.base, self.altura, self.calcular_hipotenusa()
    if math.isclose(a, b) and math.isclose(b, c):
        print("Es un triángulo equilátero")
    elif not math.isclose(a, b) and not math.isclose(a, c) and not
math.isclose(b, c):
        print("Es un triángulo escaleno")
    else:
        print("Es un triángulo isósceles")

class PruebaFiguras:
    """Clase de prueba: solo contiene el punto de entrada del programa."""

    @staticmethod
    def main():
        figura1 = Circulo(2)
        figura2 = Rectangulo(1, 2)
        figura3 = Cuadrado(3)
        figura4 = TrianguloRectangulo(3, 5)

        print(f"El área del círculo es = {figura1.calcular_area():.2f}")
        print(f"El perímetro del círculo es =
{figura1.calcular_perimetro():.2f}")
        print()
        print(f"El área del rectángulo es = {figura2.calcular_area():.2f}")
        print(f"El perímetro del rectángulo es =
{figura2.calcular_perimetro():.2f}")
        print()
        print(f"El área del cuadrado es = {figura3.calcular_area():.2f}")
        print(f"El perímetro del cuadrado es =
{figura3.calcular_perimetro():.2f}")
        print()
        print(f"El área del triángulo es = {figura4.calcular_area():.2f}")
        print(f"El perímetro del triángulo es =
{figura4.calcular_perimetro():.2f}")
        print(f"La hipotenusa del triángulo es =
{figura4.calcular_hipotenusa():.2f}")
        figura4.determinar_tipo_triangulo()

if __name__ == "__main__":
    PruebaFiguras.main()

```

Resultado de código ejecutado

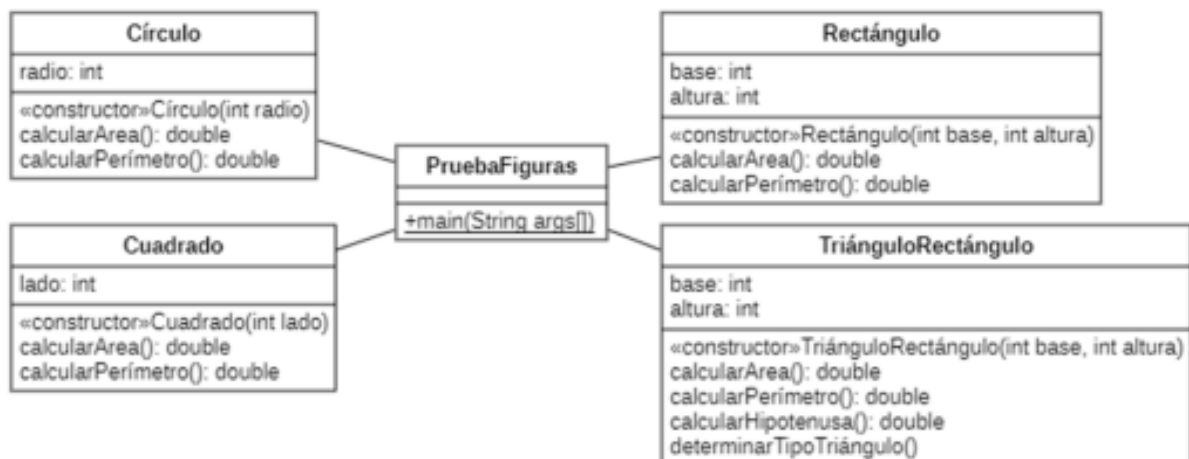
```
... El área del círculo es = 12.57
    El perímetro del círculo es = 12.57

    El área del rectángulo es = 2.00
    El perímetro del rectángulo es = 6.00

    El área del cuadrado es = 9.00
    El perímetro del cuadrado es = 12.00

    El área del triángulo es = 7.50
    El perímetro del triángulo es = 13.83
    La hipotenusa del triángulo es = 5.83
    Es un triángulo escaleno
```

Diagrama de clases



URL: https://colab.research.google.com/drive/1jNZpkG52_W6D0KFpXkx28wW69ZGgE-wG

Ejercicio 2.5. Definición de métodos con parámetros

Los métodos de una clase pueden tener parámetros, los cuales son datos que se requieren para la ejecución del método. Un método puede no contener ningún parámetro, un solo parámetro o muchos parámetros. Se recomienda que la cantidad de parámetros sea la adecuada para el correcto funcionamiento del método, es decir, que no tenga demasiados parámetros.

Python

```
"""
```

Ejercicio 2.5 - Definición de métodos con parámetros

Clase CuentaBancaria: consignar y retirar reciben un parámetro (valor).

```
"""
```

```
from enum import Enum
```

```

class TipoCuenta(Enum):
    AHORROS = "Ahorros"
    CORRIENTE = "Corriente"

class CuentaBancaria:
    """Define objetos que representan una cuenta bancaria."""

    def __init__(self, nombres_titular: str, apellidos_titular: str,
                 numero_cuenta: int, tipo_cuenta: TipoCuenta):
        # El saldo NO se recibe como parámetro: toda cuenta nueva inicia en 0
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self.saldo = 0.0

    def imprimir(self) -> None:
        print(f"Nombres del titular = {self.nombres_titular}")
        print(f"Apellidos del titular = {self.apellidos_titular}")
        print(f"Número de cuenta = {self.numero_cuenta}")
        print(f"Tipo de cuenta = {self.tipo_cuenta.value}")
        print(f"Saldo = ${self.saldo:,.2f}")

    def consultar_saldo(self) -> None:
        print(f"El saldo actual es = ${self.saldo:,.2f}")

    def consignar(self, valor: float) -> bool:
        """Suma 'valor' al saldo. Devuelve True si la operación fue válida."""
        if valor > 0:
            self.saldo += valor
            print(f"Se ha consignado ${valor:,.2f} en la cuenta. "
                  f"El nuevo saldo es ${self.saldo:,.2f}")
            return True
        print("El valor a consignar debe ser mayor que cero.")
        return False

    def retirar(self, valor: float) -> bool:
        """Resta 'valor' del saldo si es positivo y no supera el saldo."""
        if 0 < valor <= self.saldo:
            self.saldo -= valor
            print(f"Se ha retirado ${valor:,.2f} de la cuenta. "
                  f"El nuevo saldo es ${self.saldo:,.2f}")
            return True
        print("El valor a retirar debe ser mayor que cero y "
              "no puede superar el saldo actual.")

```

```

        return False

def main():
    cuenta = CuentaBancaria("Pedro", "Pérez", 123456789, TipoCuenta.AHORROS)
    cuenta.imprimir()
    cuenta.consignar(200000)
    cuenta.consignar(300000)
    cuenta.retirar(400000)
    cuenta.retirar(500000) # no se permite: supera el saldo
    cuenta.consultar_saldo()

if __name__ == "__main__":
    main()

```

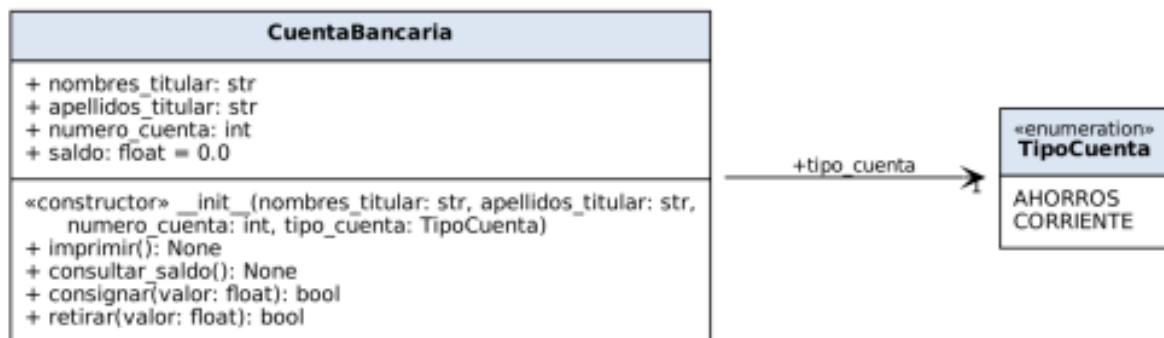
Resultado del código ejecutado

```

Nombres del titular = Pedro
Apellidos del titular = Pérez
Número de cuenta = 123456789
Tipo de cuenta = Ahorros
Saldo = $0.00
Se ha consignado $200,000.00 en la cuenta. El nuevo saldo es $200,000.00
Se ha consignado $300,000.00 en la cuenta. El nuevo saldo es $500,000.00
Se ha retirado $400,000.00 de la cuenta. El nuevo saldo es $100,000.00
El valor a retirar debe ser mayor que cero y no puede superar el saldo actual.
El saldo actual es = $100,000.00

```

Diagrama de clase



URL: https://colab.research.google.com/drive/1PtEL_Ie-TITv0TgNA_hl1L6fiPL2mnyo